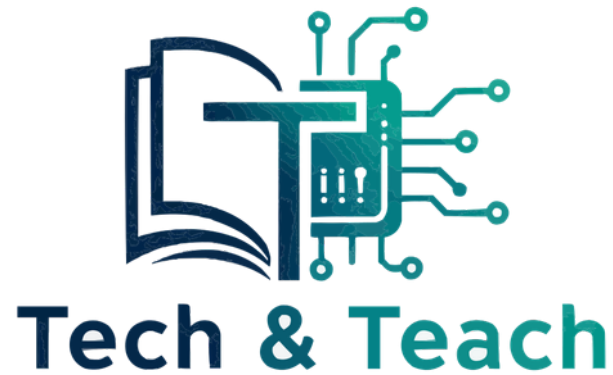




Design of 4-Bit T2 Processor

Final RTL Project

Full Stack Chip Design Program

Version 1.1

Almas Alshubrumi

Atheer Al-harbi

Mjd AL-harbi

Maha Omar

Jood Alomair

Project Overview

This project focuses on the design and implementation of a functional T2 processor using SystemVerilog. The main objective is to develop a processor capable of executing instructions, performing arithmetic and logical operations, transferring data between registers and memory, and controlling the flow of program execution.

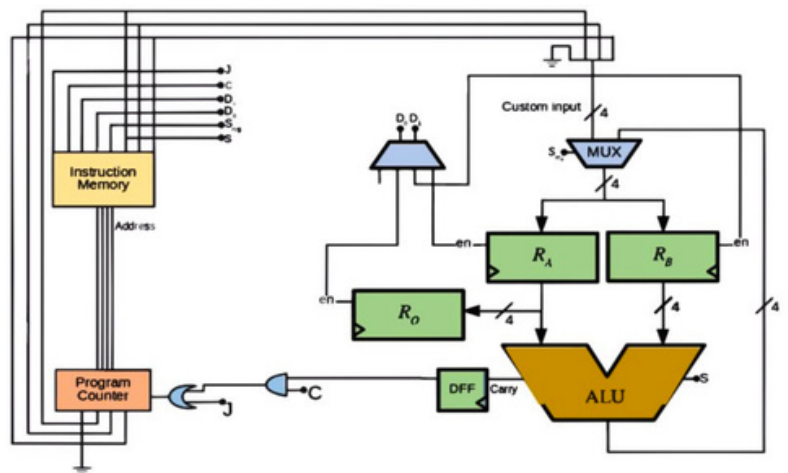
The processor consists of several essential components, including the Program Counter (PC), Instruction Memory, Data Memory, Registers, Decoder, Multiplexer (MUX), and Arithmetic Logic Unit (ALU). These components are integrated together to form the processor datapath, allowing instructions to be fetched, decoded, executed, and stored as required. The design also supports memory read and write operations and uses Carry and Zero flags to handle conditional control-flow instructions.

The complete design was implemented and tested using SystemVerilog and simulation tools. The functionality of the processor was verified by executing different instructions and observing the outputs of the registers, ALU, memory, and control signals. A Fibonacci sequence was used as the main testing application to demonstrate the processor's ability to perform repeated arithmetic operations, store intermediate results in memory, and execute instructions in the correct sequence.

Through this project, the functionality of the T2 processor was successfully demonstrated, while providing practical experience in processor architecture, datapath design, digital logic, memory organization, and SystemVerilog implementation.

Component (logic circuit) Used Phase-1

1. Instruction Memory
2. Program Counter (PC)
3. Decoder
4. Multiplexer (MUX)
5. Custom Input
6. Register R_0
7. Register R_a
8. Register R_b
9. Arithmetic Logic Unit (ALU)
10. D Flip-Flop (DFF)
11. Carry



● 1 ALU (alu.sv)

```
module Alu #(parameter WIDTH=4) (  
  input logic [WIDTH-1:0] A, // from Ra  
  input logic [WIDTH-1:0] B, // from Rb  
  input logic S,  
  output logic [WIDTH-1:0] F,  
  output logic carry  
);  
  
  logic [3:0] b_final;  
  
  assign b_final = B ^ {4{S}};  
  
  assign {carry, F} = A + b_final + S;  
  
endmodule
```

● 2 Counter (counter.sv)

```
module program_counter #(  
  parameter WIDTH = 4) (  
    input logic clk,  
    input logic rstn,  
    input logic en,  
    input logic [WIDTH-1:0] D,  
    output logic [WIDTH-1:0] Q  
  );  
  always_ff @(posedge clk or negedge rstn)  
  begin  
    if (!rstn)  
      Q <= '0;  
    else if (en)  
      Q <= D;  
    else  
      Q <= Q + 1'b1;  
    end  
  endmodule
```

● 3 Multiplexer (mux.sv)

```
module mux #(  
  parameter WIDTH = 4) (  
    input logic [WIDTH-1:0] W0,  
    input logic [WIDTH-1:0] W1,  
    input logic Sel,  
    output logic [WIDTH-1:0] F  
  );  
  assign F = Sel? W1 : W0;  
endmodule
```

● 4 Registers (registers.sv)

```
module file_registers #(
    parameter WIDTH = 4
) (
    input logic clk,
    input logic rstn,
    input logic en,
    input logic [WIDTH-1:0] data_in,
    output logic [WIDTH-1:0] data_out
);
always_ff @(posedge clk or negedge rstn)
begin
    if (!rstn)
        data_out <= '0;
    else if (en)
        data_out <= data_in;
    end
endmodule
```

● 5 InstructionMemory(instruction_memory.sv)

```
module instruction_memory #(
    parameter IMEM_DEPTH = 16,
    parameter FILE_NAME =
        "C:\\Users\\alfra\\Downloads\\machine_code.mem"
) (
    input logic [$clog2(IMEM_DEPTH)-1:0] address,
    output logic [7:0] instruction
);

logic [7:0] imem [0:IMEM_DEPTH-1];
assign instruction = imem[address];
initial
    $readmemb(FILE_NAME, imem);
endmodule
```

Tech & Teach

● 6 Decoder (Decoder.sv)

```
module Decoder (
    input logic D1, D0,
    output logic y0,y1,y2,y3
);
always_comb begin
    y0=(~D1 & ~D0);
    y1=(~D1 & D0);
    y2=(D1 & ~D0);
    y3= (D1 & D0);
end
endmodule
```

● 7 machine code:

```
00001000
00011001
00100000
00010000
01110000
00000000
00010100
00000100
10110010
```

● 8 Program Counter (PC.sv)

```
module PC #(
    parameter WIDTH = 4) (
    input logic clk,
    input logic rstn,
    input logic load,
    input logic [WIDTH-1:0] D,

    output logic [WIDTH-1:0] Q);

always_ff @(posedge clk or negedge rstn)
begin
    if (!rstn)
        Q <= 0;
    else if (load)
        Q <= D;
    else
        Q <= Q + 1'b1;
end

endmodule
```

● 9 Control Unit (control_unit.sv)

```
module control_unit (
    input logic [7:0] instruction,
    output logic J,
    output logic C,
    output logic D1,
    output logic D0,
    output logic Sreg,
    output logic S,
    output logic [2:0] imm
);

always_comb begin
    J = instruction[7];
    C = instruction[6];
    D1 = instruction[5];
    D0 = instruction[4];
    Sreg = instruction[3];
    S = instruction[2];
    imm = instruction[2:0];
end

endmodule
```

● 10 Datapath(datapath.sv)

```

module datapath #(
    parameter WIDTH = 4,
    parameter IMEM_DEPTH = 16,
    parameter FILE_NAME =
"C:\\Users\\alfra\\Downloads\\machine_code.me
m"
) (
    input logic clk,
    input logic rstn,
    input logic J,
    input logic C,
    input logic D1,
    input logic D0,
    input logic Sreg,
    input logic S,

    // Immediate value
    input logic [2:0] imm,
    // Outputs
    output logic [7:0] instruction,
    output logic [WIDTH-1:0] RO ,
    output logic Carry
);

// Program Counter output (Address)
logic [WIDTH-1:0] Q;
// Decoder outputs
logic en_RA;
logic en_RB;
logic en_RO;
// Register outputs
logic [WIDTH-1:0] RA_out;
logic [WIDTH-1:0] RB_out;
// MUX output
logic [WIDTH-1:0] MUX_out;
// ALU outputs
logic [WIDTH-1:0] ALU_out;
logic carry_wire;
PC #(
    .WIDTH(WIDTH)
) PC (
    .clk(clk),
    .rstn(rstn),
    .load(J | (C & Carry)),
    .D({1'b0, imm}),
    .Q(Q)
);
instruction_memory #(
    .IMEM_DEPTH(IMEM_DEPTH),
    .FILE_NAME(FILE_NAME)
) IM (
    .address(Q),
    .instruction(instruction)
);

Decoder DEC (
    .D1(D1),
    .D0(D0),
    .y0(en_RA),
    .y1(en_RB),
    .y2(en_RO),
    .y3()
);
file_registers #(
    .WIDTH(WIDTH)
) RA_reg (
    .clk(clk),
    .rstn(rstn),
    .en(en_RA),
    .data_in(MUX_out),
    .data_out(RA_out)
);
file_registers #(
    .WIDTH(WIDTH)
) RB_reg (
    .clk(clk),
    .rstn(rstn),
    .en(en_RB),
    .data_in(MUX_out),
    .data_out(RB_out)
);
file_registers #(
    .WIDTH(WIDTH)
) RO_reg (
    .clk(clk),
    .rstn(rstn),
    .en(en_RO),
    .data_in(RA_out),
    .data_out(RO)
);
Alu #(
    .WIDTH(WIDTH)
) ALU (
    .A(RA_out),
    .B(RB_out),
    .S(S),
    .F(ALU_out),
    .carry(carry_wire)
);
mux #(
    .WIDTH(WIDTH)
) MUX1 (
    .W0(ALU_out),
    .W1({1'b0, imm}),
    .Sel(Sreg),
    .F(MUX_out)
);
file_registers #(
    .WIDTH(1)
) dff1 (
    .clk(clk),
    .rstn(rstn),
    .en(1'b1),
    .data_in(carry_wire),
    .data_out(Carry)
);
endmodule

```


● 11 Top (topT.sv)

```
module topT #(
    parameter WIDTH = 4,
    parameter IMEM_DEPTH = 16,
    parameter FILE_NAME =
        "C:\\Users\\alfra\\Downloads\\machine_code.mem") (
    input logic clk,
    input logic rstn,
    output logic [WIDTH-1:0] RO,
    output logic Carry);
    // Internal signals
    logic [7:0] instruction;
    logic J;
    logic C;
    logic D1;
    logic D0;
    logic Sreg;
    logic S;
    logic [2:0] imm;
    //Datapath
    datapath #(
        .WIDTH(WIDTH),
        .IMEM_DEPTH(IMEM_DEPTH),
        .FILE_NAME(FILE_NAME))
    DP (
        .clk(clk),
        .rstn(rstn),
        .J(J),
        .C(C),
        .D1(D1),
        .D0(D0),
        .Sreg(Sreg),
        .S(S),
        .instruction(instruction),
        .RO(RO),
        .Carry(Carry),
        .imm(imm));
    // Control Unit
    control_unit CU (
        .instruction(instruction),
        .J(J),
        .C(C),
        .D1(D1),
        .D0(D0),
        .Sreg(Sreg),
        .S(S),
        .imm(imm) );
endmodule
```

● 12 Top Testbenche(topT_tb.sv)

```
timescale 1ns / 1ps
module topT_tb;

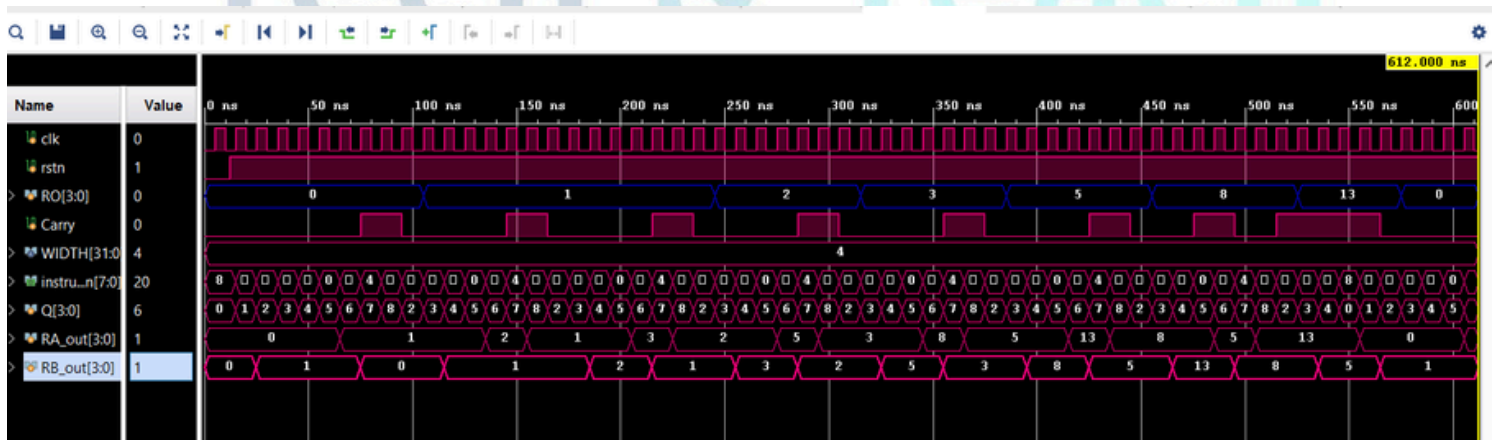
    parameter WIDTH = 4;

    logic clk ;
    logic rstn;
    logic [WIDTH-1:0] RO;
    logic Carry;

    topT #(
        .WIDTH(WIDTH) ,
        .IMEM_DEPTH(16) ,
        .FILE_NAME("C:machine_code.mem")
    ) DUT (
        .clk(clk) ,
        .rstn(rstn) ,
        .RO(RO) ,
        .Carry(Carry)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rstn = 0;
        #12;
        rstn = 1;
        #600
        $finish;
    end
endmodule
```



Phase 2

- In Phase 2, the Processor developed in Phase 1 was extended while keeping the original instructions unchanged. The main goal was to add support for Data Memory and conditional jumping based on the Zero flag.

Main Additions

- Data Memory Added a Data Memory with 4-bit data width and 8 memory locations.
- The Processor can now read data from and write data to the memory using an immediate address.

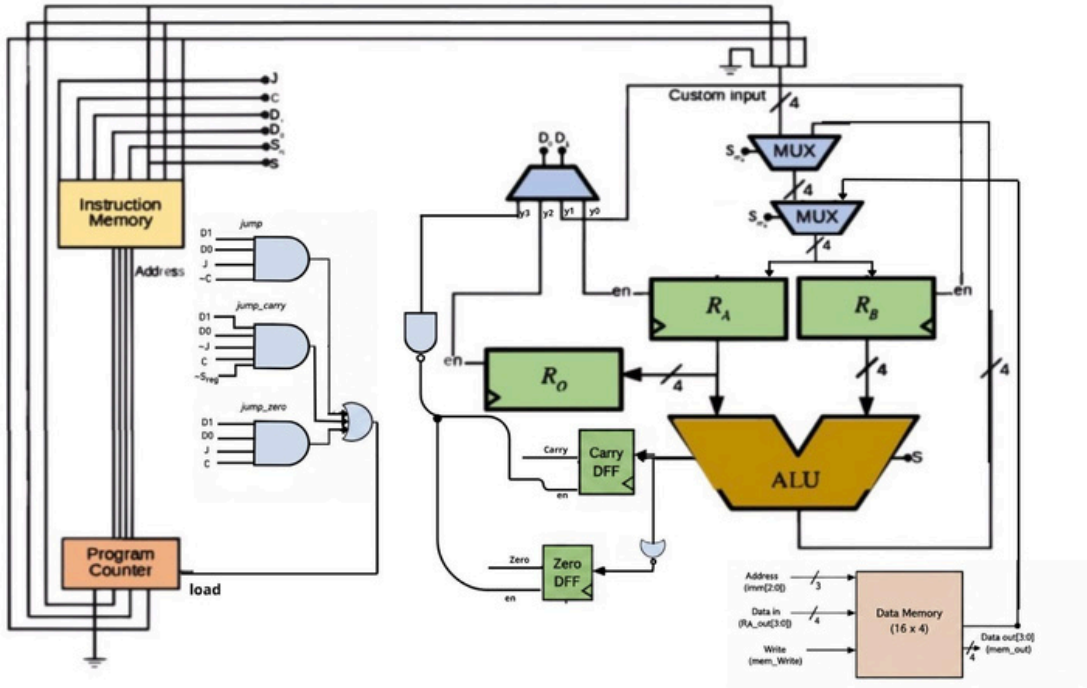
1. Memory Instructions

2. Three new instructions were added:

- $RA = [imm]$ — Loads a value from Data Memory into RA.
- $RB = [imm]$ — Loads a value from Data Memory into RB.
- $[imm] = RA$ — Stores the value of RA into a specified memory address.
- Zero Flag Added a Zero flag to detect when the ALU output is zero.
- A Zero DFF was added to store the Zero status between clock cycles.
- Jump if Zero (JZ) Added the JZ imm instruction.
- If the previous ALU result is zero, the Program Counter loads the specified immediate address and execution jumps to that location.
- Updated Jump Logic The Program Counter was updated to support:
 - Unconditional Jump
 - Jump if Carry
 - Jump if Zero

Expected Result

With these additions, the Processor can perform all the operations supported in Phase 1, while also being able to read from and write to Data Memory and perform conditional jumps based on the Zero flag. This allows the Processor to execute more advanced programs that include memory operations and conditional control flow.



1. Instruction Memory
2. Counter (PC)
3. Decoder
4. Multiplexer (MUX)
5. Register R_0
6. Register R_a
7. Register R_β
8. Arithmetic Logic Unit (ALU)
9. Register (DFF)
10. controller
11. DATA MEMORY

Phase 2

● machine_code

```
01001000 // RA=[0]
01111010 // [2]=RA
00100000 // RO=RA
01001001 // RA=[1]
01011010 // RB=[2]
00100000 // RO=RA
01111010 // [2]=RA
00000000 // RA=RA+RB
11111000 // JZ 0
01110000 // JC 0
10110100 // J 4
```

Function	J	C	D ₁	D ₀	S _{reg}	Custom Input		
						Custom Input 1	Custom Input 2	Custom Input 3
RA = [IMM]	0	1	0	0	1	imm[2]	imm[1]	imm[0]
RB = [IMM]	0	1	0	1	1	imm[2]	imm[1]	imm[0]
[IMM] = RA	0	1	1	1	1	imm[2]	imm[1]	imm[0]
JZ IMM	0	1	1	1	0	imm[2]	imm[1]	imm[0]

● Data

```
0000
0001
0000
0000
0000
0000
0000
0000
0000
```

● Data Memory(data_memory.sv)

```
module data_memory #(
    parameter WIDTH = 4,
    parameter DEPTH = 8,
    parameter FILE_NAME = "data.mem"
) (
    input logic clk,
    input logic Write,
    input logic [2:0] address,
    input logic [WIDTH-1:0] data_in,
    output logic [WIDTH-1:0] data_out
);

logic [WIDTH-1:0] dmem [0:DEPTH-1];

assign data_out = dmem[address];

always_ff @(posedge clk) begin
    if (Write)
        dmem[address] <= data_in;
end

initial
    $readmemb(FILE_NAME, dmem);

endmodule
```

● Datapath(datapath.sv)

```
module datapath #(
    parameter WIDTH = 4,
    parameter IMEM_DEPTH = 16,
    parameter IMEM_FILE = "C:\\Users\\alfra\\Downloads\\machine_code.mem",
    parameter DMEM_FILE = "C:\\Users\\alfra\\Downloads\\data.mem"
) (
    input logic clk,
    input logic rstn,
    input logic J,
    input logic C,
    input logic D1,
    input logic D0,
    input logic Sreg,
    input logic S,

    // Immediate value
    input logic [2:0] imm,
    // Outputs
    output logic [7:0] instruction,
    output logic [WIDTH-1:0] RO ,
    output logic Carry
);
// Program Counter output (Address)
logic [WIDTH-1:0] Q;
// Decoder outputs
logic en_RA;
logic en_RB;
logic en_RO;
// Register outputs
logic [WIDTH-1:0] RA_out;
logic [WIDTH-1:0] RB_out;
// MUX output
logic [WIDTH-1:0] MUX_out;
// ALU outputs
logic [WIDTH-1:0] ALU_out;
logic carry_wire;
// data memory output
logic mem_Write;
logic RA_mem;
logic RB_mem;
assign RA_mem = (!J && C && !D1 && !D0 && Sreg); //RA=[imm]
assign RB_mem = (!J && C && !D1 && D0 && Sreg); //RB=[imm]
logic [3:0] mem_out;
assign mem_Write = (!J && C && D1 && D0 && Sreg);

logic zero_wire;
logic Zero;
assign zero_wire = ~(|ALU_out);

logic jump;
logic jump_carry;
logic jump_zero;

assign jump      = D1 & D0 & J & ~C;
assign jump_carry = D1 & D0 & ~J & C & ~Sreg;
assign jump_zero  = D1 & D0 & J & C;

data_memory #(
    .WIDTH(WIDTH),
    .DEPTH(8),
    .FILE_NAME(DMEM_FILE)
```

● Datapath(datapath.sv)

```

) DM (
.clk(clk),
.Write(mem_Write),
.address(imm),
.data_in(RA_out),
.data_out(mem_out)
);
counter #(
.WIDTH(WIDTH)
) PC (
.clk(clk),
.rstn(rstn),
.load( jump|(jump_carry & Carry)|(jump_zero & Zero) ),
.D({1'b0, imm}),
.Q(Q)
);
instruction_memory #(
.IMEM_DEPTH(IMEM_DEPTH),
.FILE_NAME(IMEM_FILE)
) IM (
.address(Q),
.instruction(instruction)
);
Decoder DEC (
.D1(D1),
.D0(D0),
.y0(en_RA),
.y1(en_RB),
.y2(en_RO),
.y3()
);
registers #(
.WIDTH(WIDTH)
) RA_reg (
.clk(clk),
.rstn(rstn),
.en(en_RA|(RA_mem)),
.data_in((RA_mem) ? mem_out : MUX_out),
.data_out(RA_out)
);
registers #(
.WIDTH(WIDTH)
) RB_reg (
.clk(clk),
.rstn(rstn),
.en(en_RB|(RB_mem)),
.data_in((RB_mem) ? mem_out : MUX_out),
.data_out(RB_out)
);
registers #(
.WIDTH(WIDTH)
) RO_reg (
.clk(clk),
.rstn(rstn),
.en(en_RO && !C),
.data_in(RA_out),
.data_out(RO)
);

Alu #(
.WIDTH(WIDTH)
) ALU (
.A(RA_out),
.B(RB_out),
.S(S),
.F(ALU_out),
.carry(carry_wire)
);
mux #(
.WIDTH(WIDTH)
) MUX (
.W0(ALU_out),
.W1({1'b0, imm}),
.Sel(Sreg),
.F(MUX_out)
);
registers #(
.WIDTH(1)
) dff1 (
.clk(clk),
.rstn(rstn),
.en(~(D1 & D0)),
.data_in(carry_wire),
.data_out(Carry)
);
s #(
.WIDTH(1)
) dff2 (
.clk(clk),
.rstn(rstn),
.en(~(D1 & D0)),
.data_in(zero_wire),
.data_out(Zero)
);
endmodule

```

● 12 Top Testbenche(top_tb.sv)

```
`timescale 1ns / 1ps
module top_tb;

    parameter WIDTH = 4;

    logic clk ;
    logic rstn;
    logic [WIDTH-1:0] RO;
    logic Carry;

    top #(
        .WIDTH(WIDTH) ,
        .IMEM_DEPTH(16) ,

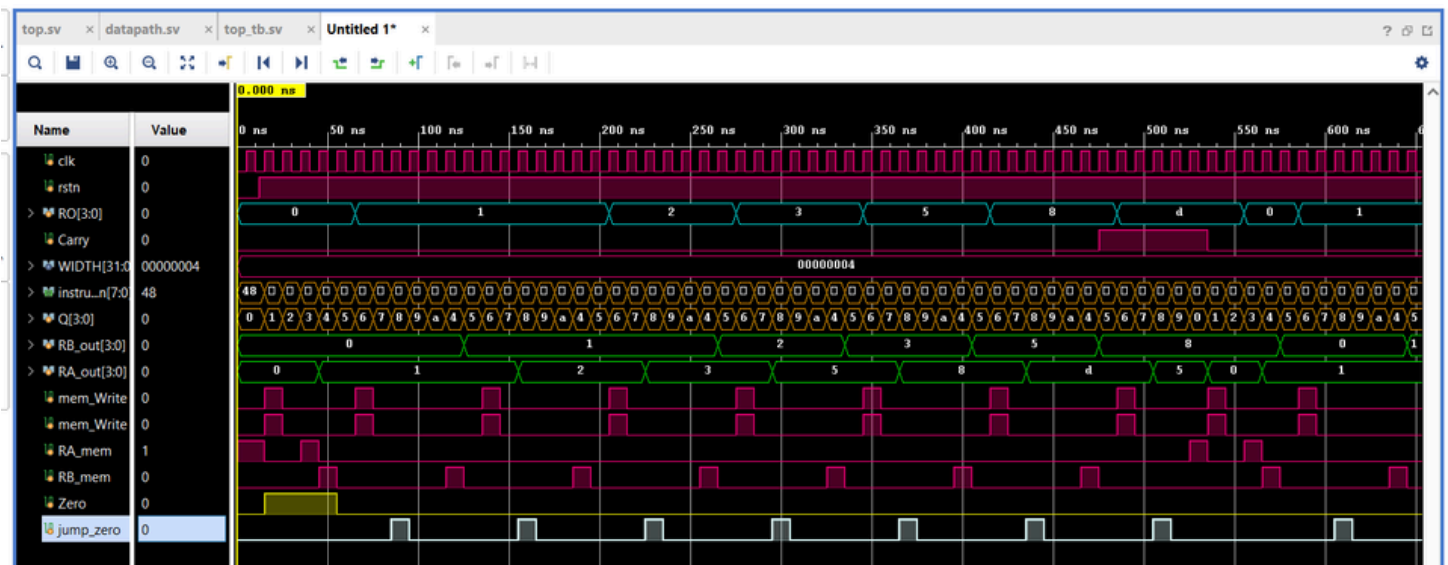
        .FILE_NAME("C:\\Users\\alfra\\Downloads\\machine_code.m
em") ,
        .DMEM_FILE
("C:\\Users\\alfra\\Downloads\\data.mem")

    ) dut (
        .clk(clk) ,
        .rstn(rstn) ,
        .RO(RO) ,
        .Carry(Carry)
    );

    always begin #5 clk = ~clk; end

    initial begin
        clk = 0;
        rstn = 0;
        #12;
        rstn = 1;

        for (int i = 0; i < 150; i = i + 1) begin
            #10;
        end
        $finish;
    end
endmodule
```



Results from FPGA

- The processor was tested on the FPGA board in both phases, and the results were correct and matched the expected values.

